

Cairn — HackMIT 2026 Spec

2026-09-19 · @Someone

Problem and thesis

A textbook is a dependency graph flattened into a line, and Cairn unflattens it. Every theorem rests on earlier definitions, results and notation, but the author had to pick one linear print order. Scrolling back to re-read Theorem 3.4 is the symptom; the lost structure is the cause.

Cairn ingests a student's whole course corpus (textbook, lecture notes, problem sets, slides), reconstructs the dependency graph across all of them, and uses that graph to make reading adaptive. Working name only, chosen for trail markers left behind so you can retrace a path.

The three claims the demo has to land:

1. **Backward.** Any invoked result can be read in place, restated in the notation of the page invoking it, narrowed to the clause actually being used.
2. **Forward.** Any definition shows where it pays off later, a direction you physically cannot scroll in.
3. **Across.** A problem set, the professor's notes and the textbook resolve to one graph despite different numbering and names.

Target user is a student in a proof-heavy course (real analysis, linear algebra, algorithms, probability). Success metric for the demo is time-to-comprehension on a late-chapter proof, measured against the same proof in a plain PDF reader.

Prior art and where the gap is

The hover tooltip alone is a solved, published problem, so the defensible ground is the corpus-level graph. Know all of this cold before judging.

Work	What it does	Why it does not cover us
ScholarPhi (CHI 2021)	Position-sensitive tooltips for terms and symbols, declutter filter, equation diagrams, auto glossary	Single research paper, researcher audience, defines symbols rather than restating theorems in local notation
Semantic Reader	Shipped descendant of ScholarPhi, inline citation cards, skimming highlights	Papers and their citation network, not textbooks; no intra-book dependency graph
TheoremGraph (2026)	11.7M statements, 18.3M dependency edges from math arXiv LaTeX source	A dataset, not a reading interface; needs LaTeX source; arXiv only
KnowTeX (2026)	Dependency graph visualiser over LaTeX theorem environments	Requires <code>.tex</code> source plus author-added annotations; static graph, no reader
TheoremKB, NaturalProofs	Large natural-language theorem dependency corpora	Corpus construction, no student-facing product
SciSpace, ChatPDF, Humata	Chat over an uploaded PDF	No structure extraction, no graph, no cross-document identity; fully commoditised

The four gaps we occupy. Arbitrary textbook PDFs with no LaTeX source. A student's own multi-document corpus with entity resolution across it. Notation instantiation, meaning the cited theorem restated in the invoking page's symbols. Reader-state adaptivity, meaning card depth that responds to what you have already read.

Framing risk. "AI-powered PDF reader" is a saturated hackathon category and a judge's first fifteen seconds will pattern-match to ChatPDF. Demo the cross-document jump first and the hover card second.

Feature spec

P0 features are the demo and must be done by hour 16. P1 features are what makes it win. P2 is cut without discussion if hour 18 arrives and P0 is not solid.

F1 — Corpus ingest (P0)

Drag any number of PDFs into one "course". Each is parsed into a text layer with per-token bounding boxes, then segmented into structural nodes: `definition`, `theorem`, `lemma`, `proposition`, `corollary`, `example`, `proof`, `notation`. Progress streams per document so the user sees nodes appearing during the 30-90 second parse. Reference anchors (spans of text like "by Theorem 3.4") are detected and stored with their bounding boxes so the reader can decorate them.

F2 — Instantiated hover card (P0)

Hovering a reference anchor opens a card in under 100ms, served entirely from cache with no model call at read time. The card shows the target statement rewritten in the notation of the invoking page, with clauses not in use collapsed behind a "show full statement" toggle. A diff strip at the top names the substitutions made, for example $A \rightarrow T, V \rightarrow \mathbb{R}^n$. Footer has page number, jump-to-source, and a "pin" that docks the card to the side rail so it survives scrolling.

The instantiation is precomputed at ingest: for every (reference anchor, target node) pair the pipeline generates one card and stores it. This is the single most important latency decision in the project.

F3 — Cross-document resolution (P0, the differentiator)

Nodes from different documents that denote the same result are merged into one canonical entity. The professor's "Rank-Nullity" and the book's "Theorem 3.22" and the pset's "the dimension theorem" become one node with three occurrences. Hovering the pset reference shows the textbook statement; the card lists all known occurrences across the corpus. Resolution is a two-stage pipeline: embedding recall, then an LLM adjudication pass over the top candidates, with a confidence score surfaced in the UI as a solid or dashed underline.

F4 — Prerequisite trace, "why am I stuck" (P1)

Select any sentence. Cairn walks the graph backward from every entity mentioned in it and returns the minimal chain of prior results needed, topologically ordered, with nodes you have already read greyed out and unread nodes expanded. Presented as a stack trace: innermost dependency at the bottom, the line you selected at the top. Cap depth at 4 and breadth at 3 per level or it becomes unreadable.

F5 — Forward links (P1)

On any definition or theorem node, a "pays off in" list of downstream nodes ranked by how load-bearing this node is for them, computed as PageRank over the reversed dependency edges. Answers "why am I learning this" and is trivially cheap once the graph exists.

F6 — Reader state and adaptive depth (P1)

Local reading state tracks per node: seen, dwelt on, hovered N times, explicitly marked known. Card depth responds. A node you marked known renders as a one-line reminder; a node you have never encountered renders with a plain-language gloss above the formal statement. Nodes hovered three or more times surface in a **Weak spots** panel, exportable as Anki-format CSV.

F7 — Corpus map (P1)

A force-directed view of the whole corpus graph, coloured by document, with reading progress overlaid as node opacity. Clicking a node jumps to its canonical occurrence. This is the screenshot that goes in the submission.

F8 — Voice reading companion (P2, sponsor-driven)

Push-to-talk over the current page. "What was that theorem again?" resolves against the nodes visible in the viewport rather than the whole corpus, so it answers in one turn without retrieval ambiguity. Speaks back the instantiated statement.

F9 — Hypothesis gap flags (P2, high risk)

When a proof applies a theorem, check that each hypothesis is discharged somewhere earlier in the visible chain. Flag unmet ones as a margin warning. Impressive if it works, cut without hesitation if the graph is noisy.

F10 — Shared corpus annotations (P2, unlocks the Meta track)

A course corpus can be joined by classmates. Aggregate hover counts become a heatmap of where the class is stuck; a "4 classmates also stopped here" marker appears in the margin. Requires auth and a shared store, so this is a genuine scope decision to make before hour 4, not after.

Architecture and stack

One Next.js app, one Python worker, one Postgres, one Elasticsearch. Everything expensive happens at ingest; the reader is a cache lookup.

```

flowchart LR
    U[PDF upload] --> P[Parse worker<br/>PyMuPDF]
    P --> S[Segment<br/>regex + Haiku]
    S --> E[Edge extract<br/>4 extractors]
    E --> DB[(Postgres<br/>nodes, edges)]
    E --> ES[(Elasticsearch<br/>hybrid index)]
    ES --> R[Resolver<br/>Sonnet adjudication]
    R --> DB
    DB --> C[Card baker]
    C --> DB
    DB --> W[Next.js reader]

```

Ingest pipeline in five stages

1. **Parse.** PyMuPDF extracts text spans with bounding boxes and font metadata. Font size and weight changes are the cheapest reliable signal for where a theorem environment starts and ends in a LaTeX-produced PDF.
2. **Segment.** A regex pass catches numbered environments (Theorem 3.4. , Definition 2.1) and gets roughly 80% of nodes for free. GPT-5.6 Luna cleans boundaries and handles unnumbered environments, batched 20 nodes per call.
3. **Edge extract.** Four extractors, cheapest first, matching the approach TheoremGraph validated at scale: deterministic (explicit Theorem 3.4 references), heuristic (the previous lemma , above), notation (a symbol defined in node X used in node Y), and LLM (named results like the spectral theorem). Only spans no earlier extractor claimed reach the LLM.
4. **Resolve.** Cross-document entity resolution, described in F3.
5. **Bake.** For every reference anchor, generate the instantiated card and store it. Parallelised, ~40% of total ingest cost.

Technology choices

Layer	Choice	Job it does	Track it serves
PDF parse	PyMuPDF	Text spans with bboxes and font metadata; fast and no OCR dependency	—
Segmentation and edges	GPT-5.6 Luna (\$0.20/\$1.20)	High-volume cheap classification over batched spans	Token Company
Instantiation and resolution	GPT-5.6 Sol (\$4/\$20)	The two calls where quality is visible to a judge	—
Prompt caching	OpenAI cached input	Book-level context reused across every batched call in a document	Token Company
Search and resolution recall	Elasticsearch	Hybrid BM25 plus OpenAI embeddings over node statements; the recall stage of entity resolution and the corpus search bar	Elastic
Graph and state	Postgres with <code>ltree</code> and recursive CTEs	Nodes, edges, occurrences, reader state; ancestor walks for F4 run in SQL	—
Frontend	Next.js 15, React 19, Tailwind	App shell, API routes, server components for the corpus map	—
PDF rendering	PDF.js with a custom text layer	Bbox-anchored overlays for reference decoration and hover targets	—
Graph view	D3 force simulation	F7 corpus map	—
Realtime ingest progress	Server-sent events	Nodes appear live during parse, which reads as fast even when it is not	—

Layer	Choice	Job it does	Track it serves
Voice	Deepgram Nova (STT) plus Aura (TTS)	F8 push-to-talk companion	Deepgram
Storage	Postgres large objects or S3-compatible local MinIO	Original PDFs	—
Deploy	Vercel for web, Railway for worker and Postgres, Elastic Cloud trial	Live URL for judging	—

Cost architecture, deliberately

This is a real engineering constraint, not a sponsor bolt-on. A 400-page textbook is roughly 250k tokens. Naive per-reference LLM calls at read time would be both slow and expensive. The design cuts it four ways: deterministic extractors handle roughly 70% of edges at zero token cost; Luna handles bulk classification at a twentieth of Sol's price; cached input reuses the chapter context across batched calls at 90% off; baked cards mean the read path makes zero model calls. Instrument all four and report measured before-and-after numbers. Skip the Batch API despite its 50% discount, since it processes within 24 hours, which is the whole event. Pull current rates at the venue: Sol is on promotional pricing through at least 21 November 2026 and the lineup moved twice this summer.

Explicit non-goals

No OCR of scanned books. No handwriting. No arbitrary-website browser extension. No mobile. No account system unless F10 is greenlit before hour 4.

Sponsor track strategy

Seven submissions, of which five cost under an hour of incremental work. Do not chase more than this; a thin build spread across ten tracks loses all ten.

Track	Fit	Incremental work	Cost	Framing
Dropbox	Primary	None beyond F3	0h	Their prompt names turning class materials into a personalised tutor and reorganising content as relationship graphs. F3 plus F7 is that, literally
Elastic	Strong	Use ES for resolution recall and the search bar rather than pgvector	~1h	Messy multi-document corpus, hybrid retrieval turns it into a navigable graph
The Token Company	Strong	Instrumentation and a measured comparison	~1h	Four-layer cost reduction with before-and-after numbers on a real 400-page book
Deepgram	Good	F8	~2h	Voice companion for reading with your hands on paper
Long Lake	Good	None, pure framing	0h	Cairn never invents content, it points at the reader's own book with a page citation. Grounded output is the skeptic's objection answered directly
Ramp	Lottery	None	0h	Saves students time. Free to submit
Meta	Conditional	F10 plus auth	~4h	Class-level stuck heatmap. Biggest prize in the doc, but this is a real fork, decide before hour 4

Track	Fit	Incremental work	Cost	Framing
OpenAI	Strong	None; already the whole stack. Use Codex during the build	0h	Judged on how creatively the API powers the experience and how Codex helped you build it. Credits go only to challenge submitters
Cognition	Process-only	Build with Devin	0h	Qualifies by how you built, orthogonal to the idea

Deliberately skipped. Visa, Maximor, Arrowstreet, GiveCampus, Regeneron, Voloridge, ASUS, Arduino, Espressif, Dimensional, SpaceXAI. All require specific domains, datasets or hardware. Warp wants developer tooling and would need a pivot to codebases or API docs.

Submission hygiene. Each track wants its own write-up. Write six short framings from one shared body of text at hour 21, not at hour 23.5.

Shared contracts

Three Claude Code sessions can only run autonomously if the seams are frozen before any of them starts. Commit this section as `CONTRACTS.md` at hour 0 and treat changes to it as a synchronous decision requiring all three.

Repo layout

```
cairn/
  packages/contracts/      # types + zod schemas + fixtures. Session A owns.
  apps/worker/             # Python ingest. Session A owns.
  apps/web/               # Next.js. Session B owns.
  apps/web/app/api/       # API routes. Session B owns shape, C owns handlers
under /api/intel/*
  packages/intel/         # resolution, instantiation, prompts. Session C
owns.
  fixtures/               # golden corpora + expected outputs. Shared,
append-only.
```

Ownership is strict. A session that needs a change in another's directory writes it as an issue in `INTEGRATION.md` and works around it with a stub. No cross-directory commits.

Core types

```
{
  "Node": {
    "id": "uuid",
    "doc_id": "uuid",
    "kind":
"definition|theorem|lemma|proposition|corollary|example|proof|notation",
    "label": "Theorem 3.4",
    "title": "Rank-Nullity",
    "statement_md": "Let  $T: V \rightarrow W$  be linear...",
    "clauses": [{"id": "i", "text": "..."}],
    "symbols": [{"sym": "T", "role": "linear map"}],
    "page": 112,
    "bbox": [72.0, 340.5, 468.0, 412.0],
    "entity_id": "uuid|null",
    "confidence": 0.0
  },
  "Edge": {
    "src": "node uuid",
    "dst": "node uuid",
    "kind": "depends_on|uses_notation|specialises|restates",
    "extractor": "deterministic|heuristic|notation|llm",
    "confidence": 0.0
  },
  "Anchor": {
    "id": "uuid",
    "doc_id": "uuid",
    "page": 340,
    "bbox": [0, 0, 0, 0],
    "surface": "by Theorem 3.4",
    "target_node_id": "uuid|null",
    "target_entity_id": "uuid|null",
    "card_id": "uuid|null"
  },
  "Card": {
    "id": "uuid",
    "anchor_id": "uuid",
    "headline": "Rank-Nullity, clause (ii)",
```

```

    "instantiated_md": "dim ker T + dim im T = dim V",
    "full_md": "...",
    "substitutions": [{"from": "A", "to": "T"}],
    "gloss": "plain-language one-liner for unread nodes",
    "source": {"doc_id": "uuid", "page": 112}
  }
}

```

API surface

Route	Owner	Contract
POST /api/corpus	B	Creates a corpus, returns {corpus_id}
POST /api/corpus/:id/docs	B	Multipart upload, enqueues worker job
GET /api/corpus/:id/events	B	SSE stream of {stage, doc_id, nodes_done, total}
GET /api/doc/:id/anchors?page=N	B	Anchors for one page, the reader's hot path
GET /api/card/:anchor_id	B	Baked card, served from Postgres, no model call
POST /api/intel/trace	C	{doc_id, selection} to an ordered dependency chain
POST /api/intel/resolve	C	Entity resolution batch, called by the worker
GET /api/intel/forward/:entity_id	C	Ranked downstream nodes
GET /api/search?q=	C	Elasticsearch hybrid query

Fixtures, the thing that actually unblocks parallelism

Before anyone writes real code, Session A commits `fixtures/golden/` containing hand-written JSON for one chapter of one book: 20 nodes, 30 edges, 15 anchors, 15 cards. Sessions B and C build against these fixtures from minute one and never wait on the

parser. A `USE_FIXTURES=1` env var makes every API route serve from `fixtures/` instead of the database. Keep that flag working to the end; it is also the demo fallback if the live parser breaks.

Conventions

Postgres via a single `schema.sql` in `packages/contracts`, applied idempotently. All model calls go through one `packages/intel/llm.ts` wrapper that logs tokens and cost to a `llm_calls` table, which is where the Token Company numbers come from. Branch per session (`sess/a-ingest`, `sess/b-reader`, `sess/c-intel`), merge to `main` only at the three integration checkpoints.

Session A — Ingestion and graph construction

Owns `apps/worker/`, `packages/contracts/`, `fixtures/`. **Deliverable** a command that takes a directory of PDFs and produces a populated Postgres with nodes, edges and anchors. **Never touches** React or prompts for instantiation.

Each phase ends with an executable acceptance test. Do not advance until it passes; if a phase overruns its budget by 50%, ship the fallback named in that phase and move on.

Phase A0 — Contracts and fixtures (45 min)

Write `packages/contracts/types.ts` with zod schemas for Node, Edge, Anchor and Card exactly as specced. Write `schema.sql`. Hand-author `fixtures/golden/analysis-ch3.json` covering 20 nodes, 30 edges, 15 anchors and 15 cards from a real chapter, and `fixtures/golden/pset4.json` with 6 nodes that deliberately restate three of the chapter's theorems under different names. Commit and push to `main` immediately; both other sessions are blocked on this and nothing else.

Accept: `pnpm test:contracts` validates every fixture against its schema.

Phase A1 — Parse layer (90 min)

PyMuPDF extraction into an intermediate `Span` record: text, page, bbox, font name, size, weight, line index. Group spans into paragraphs by vertical gap and indentation. Emit `parsed/<doc_id>.jsonl`.

Accept: on three test PDFs, span count is within 2% of the PDF's own text layer and no page returns zero spans. Log a per-document quality score; anything under 0.8 is flagged as an unsupported PDF in the UI rather than silently producing garbage.

Fallback: if a book fails, drop it from the demo set rather than fixing the parser.

Phase A2 — Segmentation (2h)

Regex pass first: numbered environment headers, `Proof.` markers, `□ / ■` terminators. Then GPT-5.6 Luna over each candidate block, batched 20 at a time, returning kind, title, clause decomposition and the symbols the node introduces. Prompt-cache the chapter context. Write nodes to Postgres.

Accept: on the golden chapter, precision and recall against the hand-authored fixture both exceed 0.85. Print a confusion table by kind. This is the number that gates everything downstream.

Phase A3 — Anchor detection (75 min)

Scan all text for reference surfaces: explicit (`Theorem 3.4` , `Lemma 2.1(ii)`), soft (`the previous lemma` , `as above`), and named (`the spectral theorem`). Record each with its bbox so Session B can decorate it. Resolve explicit ones locally by label match; leave soft and named ones with `target_node_id: null` for Session C.

Accept: every anchor in the golden fixture is found, bboxes round-trip through the reader's coordinate transform within 2px, and explicit resolution accuracy exceeds 0.95.

Phase A4 — Edge extraction (90 min)

Build the four extractors in cost order. Deterministic from resolved anchors. Heuristic from soft anchors plus proximity. Notation by matching symbols introduced in node X against symbol usage in node Y. LLM only for spans no earlier extractor claimed. Tag every edge with its extractor so the Token Company write-up can quantify what each layer saved.

Accept: graph is acyclic after removing `restates` edges; every theorem in the golden chapter has at least one inbound or outbound edge; edge F1 against fixture exceeds 0.75.

Phase A5 — Pipeline hardening (60 min)

SSE progress events at every stage boundary. Idempotent re-ingest keyed on file hash. A `--fixtures` mode that skips parsing entirely. Pre-ingest the four demo documents and snapshot the database to `fixtures/demo.dump` so the demo never depends on a live parse.

Accept: `make demo-db` restores a working corpus from scratch in under 30 seconds on a laptop with no network.

Phase A6 — Buffer and support (remainder)

Stop building. Support B and C, improve segmentation on whichever demo document looks worst on screen, and own the database snapshot through the final integration.

Session B — Reader frontend and interaction

Owns `apps/web/` including API route shells. **Deliverable** a reader that renders any ingested PDF with live hover cards, the trace panel and the corpus map. **Runs on fixtures from minute one and never waits on Session A past A0.**

This session carries the demo. Visual quality is not decoration here; a judge's verdict is formed in the first ten seconds of seeing the screen.

Phase B0 — Shell and fixture server (60 min)

Next.js app, Tailwind, dark-first theme. Every API route implemented as a thin handler that returns `fixtures/golden/*` when `USE_FIXTURES=1`. Corpus list page, document upload page, reader route `/read/:doc_id`.

Accept: `USE_FIXTURES=1 pnpm dev` serves a reader page with fixture data and no backend running.

Phase B1 — PDF rendering and coordinate transform (2h)

PDF.js canvas render with a custom absolutely-positioned overlay layer. The critical piece is a bidirectional transform between PDF user-space coordinates and viewport pixels that survives zoom and window resize. Render fixture anchor bboxes as coloured rectangles first and verify alignment visually before building anything on top.

Accept: anchor rectangles sit on their text at 50%, 100% and 200% zoom, and after a window resize, on every page of the demo document.

Risk note: this phase is the likeliest to overrun. If it is not working at the 3h mark, fall back to rendering the PDF text layer as selectable HTML and anchoring by character offset rather than bbox.

Phase B2 — Hover card (2h)

Decorate resolved anchors with a subtle underline, dashed where confidence is below 0.7. Hover opens a card after a 120ms delay, positioned to stay in viewport, dismissed on mouse-out with a 300ms grace so the user can move into it. Card renders the instantiated statement with KaTeX, the substitution strip, the collapsed-clause toggle, page citation, jump-to-source and pin. Pinned cards dock to a right rail and stack.

Accept: hovering any of the 15 fixture anchors opens a correct card in under 100ms with no layout shift, and pinning three cards leaves all three readable.

Phase B3 — Ingest experience (75 min)

Drag-and-drop upload, SSE subscription, live progress showing nodes appearing per document as they are extracted. This screen is on camera for 20 seconds of the demo, so make it good: a growing list of detected theorems with their labels, not a progress bar.

Accept: uploading four PDFs streams per-document progress and lands on the reader when complete, with a graceful state for a document that fails quality checks.

Phase B4 — Trace panel and forward links (90 min)

Text selection triggers a floating "why am I stuck" affordance. Clicking calls `/api/intel/trace` and renders the returned chain as a vertical stack-trace, innermost at the bottom, read nodes collapsed and greyed, unread nodes expanded. Forward links render as a "pays off in" section inside any card opened on a definition.

Accept: works against a hand-written trace fixture before Session C's endpoint exists; swap to live with no component changes.

Phase B5 — Corpus map (90 min)

D3 force layout over the entity graph. Colour by source document, size by PageRank, opacity by read state. Click to jump. Hover to highlight the ego network at depth 2. Pan and zoom.

Accept: renders 200 nodes at 30fps or better, and the four-document demo corpus is visually legible without manual layout tweaking.

Phase B6 — Polish (remainder)

Weak-spots panel, keyboard shortcuts, empty and error states, a landing screen with one sentence and a demo-corpus button. Then freeze and rehearse the demo path repeatedly, fixing only what breaks on that path.

Session C — Intelligence, cross-document resolution and sponsor integrations

Owns `packages/intel/`, `apps/web/app/api/intel/*`, the Elasticsearch index, the cost instrumentation and the voice layer. **Deliverable** everything that makes the graph smart plus three of the six sponsor submissions.

This session has the most independent surface area and the least UI risk, so it absorbs the sponsor work.

Phase C0 — LLM wrapper and cost ledger (45 min)

One `callModel()` in `packages/intel/llm.ts`. Every call logs model, input and output tokens, cache reads and writes, latency and unit cost into a `llm_calls` table with a stage tag. Build this before any prompt, because retrofitting instrumentation at hour 20 is how the Token Company submission gets dropped.

Accept: a script prints total cost grouped by stage, and a synthetic run of 50 calls is logged accurately.

Phase C1 — Instantiation prompt (2h)

Given the target node's statement, the invoking paragraph, and the symbol tables of both, produce a Card: which clause is in use, the statement rewritten in local notation, the substitution list, and a one-line plain gloss. Output strict JSON. Build a 20-case eval set from the golden fixture and iterate against it, not against vibes.

Accept: on the eval set, clause selection is correct in at least 17 of 20 and no substitution introduces a symbol absent from the invoking page. This is the quality bar the whole demo rests on.

Phase C2 — Elasticsearch index and hybrid search (90 min)

Index every node with statement text, label, title, symbols, document and kind. Hybrid query combining BM25 over text with a dense vector over OpenAI statement embeddings, RRF-fused. Expose `/api/search`.

Accept: searching `rank nullity` returns the right node top-1 across all four demo documents even where the phrase never appears verbatim.

Phase C3 — Cross-document entity resolution (2h)

For each unresolved node, retrieve the top 8 candidates from Elasticsearch, then adjudicate with one batched Sol call returning `same | different | specialisation` plus confidence. Union-find the positives into entities. Resolve soft and named anchors against entities rather than nodes.

Accept: on the golden pset fixture, all three deliberately-restated theorems resolve to their textbook nodes with confidence above 0.8, and there are zero false merges among the 20 chapter nodes.

Phase C4 — Trace and forward link endpoints (90 min)

`/api/intel/trace` takes a selection, extracts mentioned entities, walks the graph backward with a recursive CTE capped at depth 4 and breadth 3, filters against reader state, and returns an ordered chain with a one-line reason per hop.

`/api/intel/forward/:entity_id` returns downstream nodes ranked by PageRank over reversed edges.

Accept: tracing a known hard proof in the demo corpus returns a chain a human agrees with, in under 800ms.

Phase C5 — Cost reduction and the measurement (75 min)

Turn on cached input across all batched calls, keeping the shared chapter prefix stable so it actually hits. Add a content-hash cache so re-ingesting a document costs nothing. Add batch-size tuning. Then run the full pipeline twice on the same book, once with every optimisation disabled and once enabled, and produce a table of tokens and dollars by stage. That table is the Token Company submission.

Accept: measured end-to-end reduction of at least 60% with the quality evals from C1 and C3 unchanged.

Phase C6 — Voice companion (90 min, cut first if behind)

Deepgram streaming STT on push-to-talk, scoped to entities visible in the current viewport. One Sonnet call with that narrow context, answer spoken back with Deepgram TTS. Barge-in supported.

Accept: asking "what was that theorem again" while a page is open returns a spoken correct answer in under 3 seconds.

Phase C7 — Sponsor write-ups (60 min, hour 21)

Draft all seven submissions from one shared body: Dropbox, Elastic, Token Company, OpenAI, Deepgram, Long Lake, Ramp. The OpenAI one needs a concrete line on how Codex changed your process, not just what you shipped. Each gets its own framing paragraph and its own screenshot. Do not leave this to the last 30 minutes.

Timeline, integration and risk

Three integration checkpoints, all three sessions merging to `main` and running the demo path end to end. Between checkpoints, sessions do not block on each other.

Hour	Milestone	Gate
0-1	A0 contracts and fixtures pushed	B and C unblocked
1-6	A1-A2 parse and segment · B0-B1 shell and coordinates · C0-C1 wrapper and instantiation	
6	Checkpoint 1	Fixture-backed reader shows a real instantiated card
6-12	A3-A4 anchors and edges · B2-B3 cards and ingest UI · C2-C3 search and resolution	
12	Checkpoint 2	Live parse of one real book renders end to end. Hard gate: if this fails, cut to fixtures for the demo and say so in the write-up
12-17	A5 hardening · B4-B5 trace and map · C4-C5 endpoints and cost	

Hour	Milestone	Gate
17	Checkpoint 3	Feature freeze. Everything after this is polish, rehearsal and submissions
17-21	B6 polish · C6 voice · demo corpus finalised	
21-23	C7 write-ups, demo video, rehearse five times	
23-24	Submit. Buffer for the thing that breaks	

Risk register

Risk	Likelihood	Blast radius	Mitigation
PDF coordinate transform eats a whole session	High	Demo has no hover	B1 fallback to HTML text layer with character-offset anchoring, decided at the 3h mark
Segmentation quality too low on real books	Medium	Graph is noise	Choose demo books from LaTeX-produced PDFs only; pre-vet five candidates before hour 0
Instantiation produces confident nonsense	Medium	Worst possible failure on camera	C1 eval set of 20 cases; if under bar, degrade to showing the original statement with the relevant clause highlighted and no rewriting
Entity resolution false merges	Medium	Cross-doc demo lies	Confidence threshold at 0.8, dashed underline below it, manual allowlist for the four demo documents
Live parse fails during the demo	Low	Fatal	<code>fixtures/demo.dump</code> pre-ingested; demo the upload flow on a small document only

Risk	Likelihood	Blast radius	Mitigation
Scope creep into F9 or F10	High	Nothing finishes	F10 decided before hour 4 or never; F9 is forbidden before hour 17

Cut lines, in order

Cut F9 first, then F8 (losing Deepgram), then F7 (losing the best screenshot but nothing functional), then F6, then F5. Never cut F1, F2 or F3; those three are the project. If at hour 17 F3 does not work, the honest move is to demo the single-document version and say in the write-up that cross-document resolution is implemented but below the confidence bar, rather than faking it.

Demo script and judging

The demo opens on the differentiator, not the tooltip, because the first ten seconds decide whether a judge files this under "ChatPDF clone".

Beat by beat, 2:45

- 0:00–0:15 — The pset.** Open a problem set page. "This is my actual 18.100 pset. Problem 3 says *by the dimension theorem*. I have no idea which one that is." Hover it.
- 0:15–0:35 — The jump.** The card opens showing the theorem from the *textbook*, restated in the pset's notation, with the substitution strip visible. Click through; the reader jumps to page 112 of a different document. State plainly: three documents, one graph.
- 0:35–1:05 — The hard proof.** Open a late-chapter proof. Hover two references in sequence to show clause narrowing and notation rewriting. Point at the substitution strip once and say what it means.
- 1:05–1:30 — Why am I stuck.** Select the worst line in the proof. Show the four-step dependency chain, two greyed out because they were read, two expanded.
- 1:30–1:50 — The map.** Open the corpus map. Four colours, one connected graph, reading progress as opacity. This is the screenshot.
- 1:50–2:10 — Ingest.** Drag in a fresh short PDF and let the node list stream. Proves it is not hardcoded.

7. **2:10–2:30 — The cost table.** Twenty seconds on the measured reduction. Judges respect a team that measured something.
8. **2:30–2:45 — Close.** One sentence on who it is for and what breaks without it.

The prior-art answer, rehearsed

A judge will ask whether this is Semantic Reader. Answer it as a strength, not a defence. ScholarPhi did position-sensitive tooltips for one research paper and shipped into Semantic Reader; TheoremGraph built dependency graphs from arXiv LaTeX source at 18 million edges. Neither works on a textbook without LaTeX, neither spans a student's own multi-document corpus, and neither restates a theorem in the notation of the page invoking it. Knowing the literature cold converts the most dangerous question at the table into the best moment.

Submission checklist

- ☐ Public repo, MIT licensed, README with architecture diagram and a one-command demo setup
- ☐ 2–3 minute demo video, screen recording with voiceover, uploaded before the deadline not at it
- ☐ Live deployment URL with the demo corpus pre-loaded
- ☐ Seven sponsor write-ups drafted at hour 21, each with its own framing and screenshot
- ☐ Cost comparison table exported as an image for the Token Company submission
- ☐ Eval numbers from C1 and C3 in the README, including the ones that are mediocre
- ☐ Devpost entry with the prior-art paragraph included rather than hidden

Sources

- [ScholarPhi, CHI 2021](#)
- [The Semantic Reader Project](#)
- [TheoremGraph, 2026](#)
- [KnowTeX, 2026](#)
- [HackMIT 2026 sponsor challenges](#)